Deadlock Monitor

To monitor Deadlocks properly, you need to take a sample of Deadlocks that occur within a specified amount of time and total them up. The big sort of roadblock there is that deadlocks are purged from the system quickly, in addition there isn't a specific deadlock cou nter you need to pull them from deadlock graphs, so in order to gather a given interval during a span of time, you need to count them periodically and store those in a table for comparison purposes, then compare the amount within a specified time frame.

The best way I have found to do this, is by scanning the deadlock graphs looking for Deadlock Victims, as all Deadlock events will have at least 1 victim and then count those events as a "Deadlock".

So for Monitoring, we'll create a very light 1 row table that stores the count on one run, then a SQL job that runs the check again and compares the difference within the two time frames and then alerts when the count exceeds the defined threshold (for example, 10 deadl ocks within the past 10 minutes).

Step 1 - Create the Deadlock Table

Here we will create the initial table. This is an extremely light 1 row table and never grows past that so it can be placed a lmost anywhere, I put mine in msdb, an alternate place could be a maintenance database as well. It also requires no cleanup as the same row is cons tantly replaced, it also does need any indexing being it's only 1 row it would be pointless to have it indexed.

```
USE [msdb];
GO

IF OBJECT_ID('dbo.DeadlockMonitorState', 'U') IS NULL
BEGIN
   CREATE TABLE dbo.DeadlockMonitorState
   (
      MonitorName      sysname      NOT NULL PRIMARY KEY,
      LastProcessedUtc datetime2(3) NOT NULL,
      LastUpdatedLocal datetime2(3) NOT NULL
   );
END;
GO

IF NOT EXISTS
(
   SELECT 1
   FROM dbo.DeadlockMonitorState
   WHERE MonitorName = N'DeadlockVictimMonitor'
)
BEGIN
   INSERT dbo.DeadlockMonitorState
   (
      MonitorName,
      LastProcessedUtc,
      LastUpdatedLocal
   )
   VALUES
   (
      N'DeadlockVictimMonitor',
      SYSUTCDATETIME(),
      SYSDATETIME()
   );
END;
GO
```

Step 2 - Create the SQL Job

The script below creates the SQL job to check the Deadlock Victims table for deadlock counts within the past 5 minutes and by default alerts when the count is greater than 2, on any database. The job is scheduled to run every 5 minutes. It does this by running the c heck once, writing to the table its findings, then on the second run it compares and updates the table row with the counts. To widen (or shorten) the timespan it checks (e.g - Amount of Deadlocks within the past 15 minutes) you would just adjust the job's schedule interval to check every 15 minutes

checks (e.g - Amount of Deadlocks within the past 15 minutes) you would just adjust the job's schedule interval to check every 15 minutes instead of 5.

You can play with the Threshold and Job Schedule to find a happy medium depending on what severity levels you want. So to cat ch smaller scenarios or pending storms every 5 minutes with a greater than or equal to 2 threshold, or if you only want to catch large s torms then set it to every 5 minutes at a threshold of 5 for example. Just to note, tighter thresholds may cause more alert noise, or trigger beni gn issues so keep that in mind.

For the highlighted parts below, You can modify the threshold at the beginning of the job step script to whatever threshold y ou want, 5 is the default, and the second highlighted part is for the email operator who the emails will be sent to if it alerts, this can be c hanged to whatever SQL Operator you have setup on the server.

```
USE [msdb];
GO

SET NOCOUNT ON;
GO

DECLARE @jobId UNIQUEIDENTIFIER;

-----------------------------------------------------------------------------
-- Drop existing job if it already exists
-----------------------------------------------------------------------------
IF EXISTS
(
   SELECT 1
   FROM msdb.dbo.sysjobs
   WHERE name = N'Deadlock Monitor'
)
BEGIN
   EXEC msdb.dbo.sp_delete_job
      @job_name = N'Deadlock Monitor',
      @delete_unused_schedule = 1;
END
GO

DECLARE @jobId UNIQUEIDENTIFIER;

-----------------------------------------------------------------------------
-- Create Job
-----------------------------------------------------------------------------
EXEC msdb.dbo.sp_add_job
   @job_name = N'Deadlock Monitor',
   @enabled = 1,
   @description = N'Alerts when >= 2 deadlock victims occur across user DBs in the last 5 minutes (system_health XE).',
   @start_step_id = 1,
   @owner_login_name = N'sa',
   @job_id = @jobId OUTPUT;

-----------------------------------------------------------------------------
-- Add Job Step 1
-----------------------------------------------------------------------------
EXEC msdb.dbo.sp_add_jobstep
   @job_id = @jobId,
   @step_id = 1,
   @step_name = N'Deadlock Monitor',
   @subsystem = N'TSQL',
   @database_name = N'master',
   @on_success_action = 1,   -- Quit with success
   @on_fail_action = 2,      -- Quit with failure
   @command = N'
SET NOCOUNT ON;
SET QUOTED_IDENTIFIER ON;
```

```
SET QUOTED_IDENTIFIER ON;
SET ANSI_NULLS ON;
SET ANSI_PADDING ON;
SET ANSI_WARNINGS ON;
SET CONCAT_NULL_YIELDS_NULL ON;
SET ARITHABORT ON;
SET NUMERIC_ROUNDABORT OFF;

BEGIN TRY
    DECLARE @MonitorName      sysname       = N''DeadlockVictimMonitor'';
    DECLARE @Threshold        int           = 2;
    DECLARE @OverlapSeconds   int           = 15;
    DECLARE @NowUtc           datetime2(3)  = SYSUTCDATETIME();
    DECLARE @NowLocal         datetime2(3)  = SYSDATETIME();
    DECLARE @LastProcessedUtc datetime2(3);
    DECLARE @ReadFromUtc      datetime2(3);
    DECLARE @NewMaxUtc        datetime2(3);
    DECLARE @RunDateTime      varchar(19)   = CONVERT(varchar(19), GETDATE(), 120);

    ------------------------------------------------------------
    -- Read and lock the monitor state row
    ------------------------------------------------------------
    SELECT
        @LastProcessedUtc = LastProcessedUtc
    FROM msdb.dbo.DeadlockMonitorState WITH (UPDLOCK, HOLDLOCK)
    WHERE MonitorName = @MonitorName;

    IF @LastProcessedUtc IS NULL
    BEGIN
        RAISERROR(''Deadlock monitor state row not found for %s.'', 16, 1, @MonitorName);
        RETURN;
    END;

    SET @ReadFromUtc = DATEADD(SECOND, -@OverlapSeconds, @LastProcessedUtc);

    ------------------------------------------------------------
    -- Stage raw victim rows
    ------------------------------------------------------------
    IF OBJECT_ID(''tempdb..#VictimsRaw'') IS NOT NULL DROP TABLE #VictimsRaw;
    CREATE TABLE #VictimsRaw
    (
        DeadlockTimeUtc   datetime2(3) NOT NULL,
        DeadlockTimeLocal datetime2(3) NOT NULL,
        DatabaseName      sysname      NULL,
        VictimID          varchar(100) NOT NULL
    );

    ;WITH XE AS
    (
        SELECT
            TRY_CAST(event_data AS xml) AS event_data
        FROM sys.fn_xe_file_target_read_file(''system_health*.xel'', NULL, NULL, NULL)
        WHERE object_name = ''xml_deadlock_report''
    ),
    Parsed AS
    (
        SELECT
            x.event_data,
            x.event_data.value(''(//event/@timestamp)[1]'', ''datetime2(3)'') AS DeadlockTimeUtc
        FROM XE x
        WHERE x.event_data IS NOT NULL
          AND x.event_data.value(''(//event/@timestamp)[1]'', ''datetime2(3)'') > @ReadFromUtc
          AND x.event_data.value(''(//event/@timestamp)[1]'', ''datetime2(3)'') <= @NowUtc
    ),
```

```
),
Victims AS
(
   SELECT
      p.DeadlockTimeUtc,
      CAST(
         p.DeadlockTimeUtc AT TIME ZONE ''UTC'' AT TIME ZONE ''Central Standard Time''
         AS datetime2(3)
      ) AS DeadlockTimeLocal,
      DB_NAME(pr.procnode.value(''(@currentdb)[1]'', ''int'')) AS DatabaseName,
      v.victimProcess.value(''(@id)[1]'', ''varchar(100)'') AS VictimID
   FROM Parsed p
   CROSS APPLY p.event_data.nodes(''//victim-list/victimProcess'') AS v(victimProcess)
   CROSS APPLY p.event_data.nodes(''//process-list/process'') AS pr(procnode)
   WHERE v.victimProcess.value(''(@id)[1]'', ''varchar(100)'')
      = pr.procnode.value(''(@id)[1]'', ''varchar(100)'')
)
INSERT #VictimsRaw
(
   DeadlockTimeUtc,
   DeadlockTimeLocal,
   DatabaseName,
   VictimID
)
SELECT
   v.DeadlockTimeUtc,
   v.DeadlockTimeLocal,
   v.DatabaseName,
   v.VictimID
FROM Victims v
WHERE v.DeadlockTimeUtc > @LastProcessedUtc;

------------------------------------------------------------
-- Deduplicate the current run
------------------------------------------------------------
IF OBJECT_ID(''tempdb..#Victims'') IS NOT NULL DROP TABLE #Victims;
CREATE TABLE #Victims
(
   DeadlockTimeUtc   datetime2(3) NOT NULL,
   DeadlockTimeLocal datetime2(3) NOT NULL,
   DatabaseName      sysname      NULL,
   VictimID          varchar(100) NOT NULL
);

;WITH Dedup AS
(
   SELECT
      vr.*,
      rn = ROW_NUMBER() OVER
      (
         PARTITION BY vr.DeadlockTimeUtc, vr.VictimID, ISNULL(vr.DatabaseName, N'''')
         ORDER BY vr.DeadlockTimeUtc
      )
   FROM #VictimsRaw vr
)
INSERT #Victims
(
   DeadlockTimeUtc,
   DeadlockTimeLocal,
   DatabaseName,
   VictimID
)
SELECT
   DeadlockTimeUtc,
```

```
   DeadlockTimeUtc,
   DeadlockTimeLocal,
   DatabaseName,
   VictimID
FROM Dedup
WHERE rn = 1;

-------------------------------------------------------------
-- Per-database counts
-------------------------------------------------------------
IF OBJECT_ID(''tempdb..#DeadlockCounts'') IS NOT NULL DROP TABLE #DeadlockCounts;
CREATE TABLE #DeadlockCounts
(
   DatabaseName sysname NOT NULL,
   VictimCount  int     NOT NULL
);

;WITH MonitoredDBs AS
(
   SELECT d.name AS DatabaseName
   FROM sys.databases d
   WHERE d.database_id > 4
     AND d.state_desc = ''ONLINE''
     AND d.is_read_only = 0
),
VictimCounts AS
(
   SELECT
      v.DatabaseName,
      COUNT(*) AS VictimCount
   FROM #Victims v
   WHERE v.DatabaseName IS NOT NULL
   GROUP BY v.DatabaseName
)
INSERT #DeadlockCounts
(
   DatabaseName,
   VictimCount
)
SELECT
   m.DatabaseName,
   ISNULL(vc.VictimCount, 0)
FROM MonitoredDBs m
LEFT JOIN VictimCounts vc
   ON vc.DatabaseName = m.DatabaseName;

DECLARE @GrandTotal int =
(
   SELECT ISNULL(COUNT(*), 0)
   FROM #Victims
);

SELECT @NewMaxUtc = MAX(DeadlockTimeUtc)
FROM #Victims;

-------------------------------------------------------------
-- Send alert if threshold met
-------------------------------------------------------------
IF @GrandTotal >= @Threshold
BEGIN
   DECLARE @body nvarchar(max) =
      CONCAT(
         N''Deadlock Victim Monitor alert.'', CHAR(13), CHAR(10),
         N''Query Run Time: '', @RunDateTime, CHAR(13), CHAR(10),
```

```
            N''Query Run Time: '', @RunDateTime, CHAR(13), CHAR(10),
            N''Processing new victims since last watermark.'', CHAR(13), CHAR(10),
            N''Previous watermark UTC: '', CONVERT(nvarchar(23), @LastProcessedUtc, 121), CHAR(13), CHAR(10),
            N''Current read upper bound UTC: '', CONVERT(nvarchar(23), @NowUtc, 121), CHAR(13), CHAR(10),
            N''Threshold: '', CONVERT(nvarchar(10), @Threshold), N'' victim(s)'', CHAR(13), CHAR(10),
            N''Grand Total New Victims: '', CONVERT(nvarchar(10), @GrandTotal), CHAR(13), CHAR(10),
            CHAR(13), CHAR(10),
            N''--- PER DATABASE (victims) ---'', CHAR(13), CHAR(10)
          );

        SELECT @body =
          @body + CONCAT(
            N''DB: '', QUOTENAME(DatabaseName),
            N'' | victims='', CONVERT(nvarchar(10), VictimCount),
            CHAR(13), CHAR(10)
          )
        FROM #DeadlockCounts
        WHERE VictimCount > 0
        ORDER BY VictimCount DESC, DatabaseName;

        IF NOT EXISTS (SELECT 1 FROM #DeadlockCounts WHERE VictimCount > 0)
        BEGIN
          SET @body += N''(No per-database victims resolved from deadlock reports.)'' + CHAR(13) + CHAR(10);
        END;

        SET @body += CHAR(13) + CHAR(10) + N''--- DETAILS ---'' + CHAR(13) + CHAR(10);

        SELECT @body =
          @body + CONCAT(
            N''TimeLocal='', CONVERT(nvarchar(23), DeadlockTimeLocal, 121),
            N'' | DB='', COALESCE(QUOTENAME(DatabaseName), N''[Unknown]''),
            N'' | VictimID='', VictimID,
            CHAR(13), CHAR(10)
          )
        FROM #Victims
        ORDER BY DeadlockTimeUtc, DatabaseName, VictimID;

        DECLARE @ServerName sysname;
        SELECT @ServerName = CAST(SERVERPROPERTY(''ServerName'') AS sysname);
        DECLARE @Subject nvarchar(255);
        SET @Subject = N''SQL Deadlock Victim Alert on '' + @ServerName;

        EXEC msdb.dbo.sp_notify_operator
          @name    = N''sqlemailoperator'',
          @subject = @Subject,
          @body    = @body;
      END;

      ------------------------------------------------------------
      -- Advance watermark only after successful processing
      ------------------------------------------------------------
      UPDATE msdb.dbo.DeadlockMonitorState
      SET
        LastProcessedUtc = ISNULL(@NewMaxUtc, @LastProcessedUtc),
        LastUpdatedLocal = @NowLocal
      WHERE MonitorName = @MonitorName;

    END TRY
    BEGIN CATCH
      THROW;
    END CATCH;
    ';

    ------------------------------------------------------------------------------
```

```
-------------------------------------------------------------------------------
-- Create Schedule
-------------------------------------------------------------------------------
EXEC msdb.dbo.sp_add_schedule
   @schedule_name = N'Deadlock Monitor - Every 5 Minutes',
   @enabled = 1,
   @freq_type = 4,              -- Daily
   @freq_interval = 1,          -- Every day
   @freq_subday_type = 4,       -- Minutes
   @freq_subday_interval = 5,   -- Every 5 minutes
   @active_start_time = 000000;  -- 00:00:00

-------------------------------------------------------------------------------
-- Attach Schedule to Job
-------------------------------------------------------------------------------
EXEC msdb.dbo.sp_attach_schedule
   @job_id = @jobId,
   @schedule_name = N'Deadlock Monitor - Every 5 Minutes';

-------------------------------------------------------------------------------
-- Add Job to local server
-------------------------------------------------------------------------------
EXEC msdb.dbo.sp_add_jobserver
   @job_id = @jobId,
   @server_name = N'(LOCAL)';
GO
```

Once setup, your alert emails should look something like this...

```
Subject:
SQL Deadlock Victim Alert on SQLSERVER01

Body:

Deadlock Victim Monitor Alert

Query Run Time: 2026-01-01 10:05:00

Processing new victims since last watermark.

Previous Watermark (UTC): 2026-01-01 10:00:00
Current Window End (UTC): 2026-01-01 10:05:00

Threshold: 2 victim(s)
Total New Victims: 4

--- Per Database (Victims) ---
DB: [DatabaseA]     | victims = 3
DB: [DatabaseB]  | victims = 1

--- Details ---
TimeLocal=2026-01-01 10:01:12 | DB=[DatabaseA]    | VictimID=process1
TimeLocal=2026-01-01 10:02:45 | DB=[DatabaseA]    | VictimID=process2
TimeLocal=2026-01-01 10:03:30 | DB=[DatabaseB] | VictimID=process3
TimeLocal=2026-01-01 10:04:10 | DB=[DatabaseA]    | VictimID=process4
```

Step 3 - Check Deadlock Victim Table (Optional)

Here is a query to check the Deadlock Victims table. There isn't much value to the table other than processing/update times, but it does help with confirming the table is being updated and the SQL job is functioning properly.

```
SELECT TOP (1000) [MonitorName]
      ,[LastProcessedUtc]
      ,[LastUpdatedLocal]
  FROM [msdb].[dbo].[DeadlockMonitorState]
```